

The Algorithm Vortex

EDITED BY JEFFREY E. FOWLER AND JEFFREY D. FOWLER

THE NEW YORK UNIVERSITY LIBRARY OF THE HISTORY OF SCIENCE

AND THE UNIVERSITY OF CHICAGO PRESS



Volume 11, Number 1, 2011
ISSN 0013-788X
CODEN NYSB



Once you discover AI coding, there's no going back.

It is faster than you at a ridiculous number of things. It knows libraries you forgot existed. It can stare at a stack trace and notice something you have been ignoring for an hour. Then, five minutes later, it does something unbelievably stupid, believes the stupid thing completely and builds three more decisions on top of it.

This is the strange reality behind all the vibe-coding excitement. The machine is extremely capable, but you are still there. You check the architecture. You notice the missing case. You tell it that no, we are not redesigning the database because one button is the wrong color. You keep enough of the project in your own head to notice when the agent quietly wanders into another universe.

The previous chapter ended with a claim: as more of the search moves into the machine, human control has to move upward from individual actions toward the environment, feedback and boundaries surrounding those actions.

That sounds reasonable in prose.

I wanted to see if it survived contact with an actual problem.

Production software is almost the worst place to test it. A supposedly simple task may involve deployment, legacy systems, users, security, another team's API and a requirement nobody wrote down because everyone assumed everybody else knew it. If the agent fails, you often don't know whether the problem was intelligence, infrastructure, missing context or the fact that someone named a database column `new_status_final_2`.

I wanted something cleaner: a hard problem, but contained. Something where I could genuinely say, “figure it out,” and still have an objective way to know whether whatever came back was any good.

I call these **bounded problems**. Not easy problems. Quite the opposite. They can require serious mathematics, programming, research or design, but the boundary is unusually cooperative: you can describe the problem, give the agent enough tools to work on it and evaluate what comes back without deploying to ten million customers first.

Algorithms are almost perfect for this. The search can be brutally difficult while the evaluator remains wonderfully stupid.

And that is how I ended up spending an unreasonable amount of time packing circles into a square.

The Running Example: Circle Packing

The problem is simple enough to explain to a child. Take 26 circles and put them inside a square. None may overlap, none may cross the boundary and the circles do not have to be the same size. We want to maximize the sum of their radii.

That's the whole thing. No customers, no authentication, no stakeholder arriving after the first demo to explain that what they *really* wanted was the opposite of what they originally asked for.

Just circles.

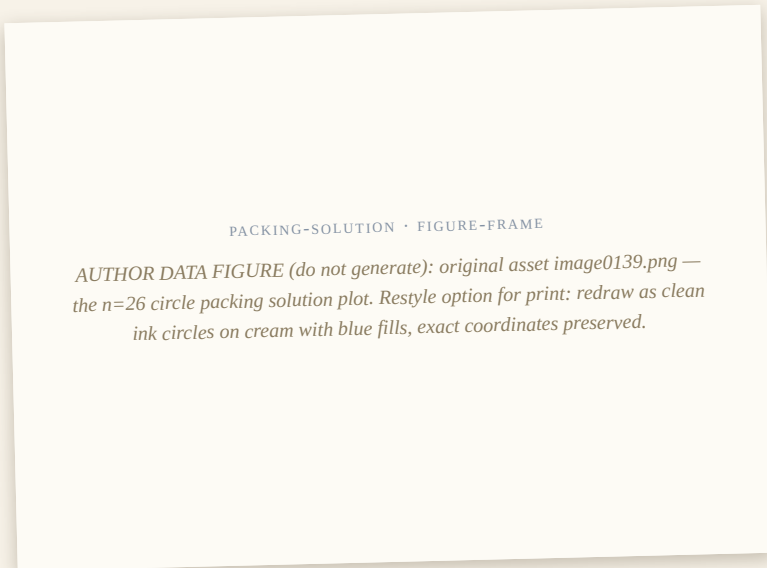
Unfortunately, the solution space is nasty.

Every circle has a position and a radius, and nearly every decision affects several others. Increase one radius and two neighbors may overlap. Move a neighbor and something else now needs to move. A packing can look almost perfect while being trapped in a configuration where every obvious improvement makes the solution invalid.

For the experiments in this chapter, we had a strong reference score around **2.635** under the evaluator we were using—the value DeepMind's AlphaEvolve reported in 2025, when it nudged the best known packing for 26 circles up from 2.634.

CITRUS · CORNER-BLEED

Corner watercolor: a wooden market crate of oranges packed improbably well, one orange of a different size wedged perfectly; a small robot with calipers measuring the gap. Warm ochre dominant.



A strong reference packing for the 26-circle objective (≈ 2.635 under our evaluator)

Figure: A strong reference packing for the 26-circle objective, scoring approximately 2.635 under our evaluator.

This is what makes the problem useful for studying autonomy. Searching is hard, but judging is cheap. The evaluator does not care whether the agent has a persuasive explanation for why two circles ought to overlap slightly in the name of geometric inclusivity. It checks the constraints and returns a score.

There is something deeply comforting about an evaluator with no personality. A candidate does not earn trust because its explanation sounds clever. It earns another round because it was exposed to something outside the model that did not care about the explanation and survived.

The experiment becomes interesting once we ask a second question:

Who is inventing the next move?

For most of the history of algorithm design, the answer was us.

Humans invented explicit algorithms. When direct algorithms were not enough, we invented optimization procedures that searched over candidate solutions. Then we invented meta-heuristics that searched more broadly. Machine learning let systems learn useful structure from data. Now language models can write and modify the search procedure itself.

A crude taxonomy helps. **Symbolic methods** give us explicit procedures, constraints and solvers: they are executable, testable and usually clear about what counts as a valid move. **Neural methods** give us learned intuition: useful structure we did not explicitly encode. **Neuro-symbolic systems** put the two in the same loop—let the learned model propose and let code, mathematics or another formal system decide what survives. The agentic step pushes one level further: increasingly, the agent can help decide which method to try, combine or abandon.

Circle packing lets us watch that handoff happen in miniature.

First Idea: Hill Climbing

If I gave you a rough packing and asked you to improve it manually, one obvious strategy would be to make small changes. Move a circle slightly, increase a radius, see whether the result is still valid, keep it if the score improves and undo it if it doesn't.

That is hill climbing:

1. Start with a valid solution.
2. Perturb a position or radius.
3. Check whether the result is valid.
4. Keep it if the score improves.
5. Repeat.

Early in the search, this works nicely. There is empty space and plenty of room to improve. Later, as the circles become tightly packed, almost every interesting move creates an overlap.

HILL-CLIMBING · FIGURE-FRAME

AUTHOR DATA FIGURE (do not generate): original asset image0140.png — hill-climbing progression panels/score curve. Keep data exact; restyle to book palette.

Hill climbing: early mutations accepted, then the search stalls

Figure: Early mutations are often accepted, but as the packing tightens, valid improvements become increasingly rare and the search stalls.

In one simple run, the score climbed from around 1.33 to roughly 2.26. That is not terrible, but it is also nowhere near 2.635.

Hill climbing is not failing because it is stupid. It is doing exactly what we asked: improving the solution immediately around it. The problem is that the current solution may live in the wrong part of the search space. Reaching a much better packing may require temporarily moving through configurations that look worse, or jumping to a structure that cannot be reached through a sequence of tiny improvements.

This matters far beyond circle packing. A system can become expert at improving the thing in front of it while never questioning whether the thing in front of it is the right thing to improve.

Here, the machine is searching—but the human still invented the search rule.

So we give the machine a bigger space.

Evolutionary Algorithms

Hill climbing puts all your evolutionary eggs in one basket. One solution

gets a very long life, and if its history leads into the wrong valley, the search inherits that history forever.

Evolutionary methods keep a **population**.

Instead of dropping one climber onto the landscape, drop a hundred. Some begin in terrible places, some find respectable hills and a few may stumble into structures the original trajectory would never have reached. The biological vocabulary—population, mutation, selection, crossover—is familiar, but the metaphor is optional. What matters is diversity: the whole search no longer inherits the assumptions of one initial guess.

For circle packing, mutation is easy enough to imagine. Move circles. Change radii. Perturb several values at once.

Almost immediately, however, we hit a practical problem. Most interesting mutations break the packing. Two circles overlap or one moves outside the square. The mutation may point toward an interesting arrangement, but the result itself is invalid.

So we added **virtual forces**. When circles overlap, imagine them repelling one another. After mutation or crossover, run a repair procedure that pushes the circles away from collisions and back inside the boundary.

This helps a lot, but notice what happened: the evolutionary algorithm did not invent virtual forces. We did.

Then we reached crossover. Suppose Parent A and Parent B both contain useful geometric structure. How do we combine them? The naive answer is to pair circle 0 from one parent with circle 0 from the other, circle 1 with circle 1, and so on.

That is usually nonsense because circle numbering is arbitrary. Two nearly identical arrangements may store corresponding circles at completely different indices.

So we used **bipartite matching crossover**. Rather than pair circles by position in an array, pair them according to their geometric role in the packing. The Hungarian algorithm gives us an efficient assignment, after which crossover has some chance of combining meaningful parts of the two parents instead of averaging unrelated circles and asking geometry for forgiveness.

CROSSOVER · FIGURE-FRAME

*AUTHOR DATA FIGURE (do not generate): original asset image0141.png —
naive vs bipartite-matching crossover diagram. Keep structure exact; restyle to
book palette.*

Naive vs. geometric crossover

Figure: Naive crossover pairs circles by array index and often destroys useful structure. Geometric matching tries to identify corresponding circles before combining the parents.

Now we can evolve a population: mutate, repair, cross, select and repeat.

EVOLUTION-RESULT · FIGURE-FRAME

*AUTHOR DATA FIGURE (do not generate): original asset image0122.png —
evolutionary strategies result. Keep data exact; restyle to book palette.*

Evolutionary search with geometric crossover

Figure: Starting around 2.08, the evolutionary search reaches roughly 2.45 in this experiment—much better than the simple hill climber, but still below our reference.

This is much stronger than hill climbing. It also makes the bottleneck clearer. Every time the search became substantially better, I had added something important. I decided we needed repair. I decided how crossover should respect geometry. I chose the representation.

The optimizer searched, but I was still inventing most of the useful moves.

MAP-Elites: Don't Kill Weird Ideas Too Early

Ordinary evolutionary search has another problem. If you maintain a hundred solutions and repeatedly keep only the highest-scoring ones, the population eventually starts looking like one large extended family. That can be excellent for exploitation and terrible for discovering a genuinely different strategy.

MAP-Elites takes a different approach. Instead of ranking every candidate on one axis and keeping only the winners, you describe solutions along a few behavioral dimensions and preserve the best candidate in different regions of that space.

For circle packing, perhaps one dimension measures symmetry and another measures how much circle sizes vary. One part of the archive may contain highly symmetric solutions. Another may contain asymmetric solutions with several large circles. Somewhere else may sit an ugly packing with a mediocre score and one strange structural idea that becomes useful five generations later.

This is **quality-diversity search**. The point is not merely to preserve the current winner, but to keep qualitatively different directions alive long enough to discover whether any of them become interesting.

I like this because optimization is often unfair to immature ideas. A new approach can initially perform badly simply because nobody has polished it yet. If the first respectable solution immediately kills everything else, the search can become impressively efficient at discovering one family of answers.

But MAP-Elites introduces another human choice: what dimensions define the archive? Symmetry? Radius variance? Number of large circles? Something topological? Something I haven't thought of?

The search had become more sophisticated, but the human was still deciding what counted as an interesting direction.

That is the invention problem.

The Invention Problem

By this point, the search machinery was fairly capable. We had hill climbing, population search, repair, geometric crossover and quality-diversity archives. We could evaluate huge numbers of candidate packings and inspect far more of the search space than any human would explore manually.

Yet every substantial conceptual jump came from somebody noticing something. Someone had to invent virtual forces. Someone had to realize that crossover should respect geometry. Someone had to choose the representation and decide which kinds of diversity were worth preserving.

Traditional search is excellent once we define the space and the legal moves. Sometimes the space and the moves are exactly the things we need to rethink.

Learned models have something to offer exactly that problem.

I once asked an image-generation model to produce a picture of a circle-packing solution. This was not a serious benchmark; I have no idea what related examples it may have encountered during training, and I can already hear Reviewer 2 clearing his throat.

I wanted to see something simpler: did the model have any useful geometric intuition about what a dense packing should look like?

Surprisingly, yes. It generated something that looked plausible. The circles had structure. The spacing looked intentional. At a glance, you could believe the model understood the problem.

Then you counted the circles.

Wrong number.

Some constraints were violated.

It was a beautiful answer to a nearby problem.

That little experiment makes the asymmetry concrete. Learned models can be remarkably good at generating plausible structure without guaranteeing that every formal requirement survives generation. A symbolic optimizer has almost the opposite personality: give it a precise representation and constraints and it will obey them, but it will not naturally

look at your representation and decide that you have been unimaginative.

The obvious temptation is to argue about which one is better. The more useful answer is: **put them in the same loop**—neural intuition and symbolic rigor.

Or, if you prefer the slightly ridiculous version: let the brain invent things and make the body prove they work.

The important move is to stop asking the model to produce the packing directly.

Ask it to write the program that produces the packing.

Let the Model Write the Solver

A candidate no longer needs to be only a list of circle positions and radii:

```
(x1, y1, r1), (x2, y2, r2), ...
```

It can be an entire program:

```
solve_circle_packing.py
```

One program may use constrained optimization. Another simulated annealing. Another a geometric construction. Another may combine a hand-designed initialization with numerical refinement.

The evaluator does not care which family produced the solution. It runs the program, checks the geometry and scores the result.

This gives the language model a much more interesting role. Rather than randomly perturbing numbers, it can read the program, form a rough theory about why it underperforms and change the algorithm. Perhaps the initialization is weak. Change the initialization. Perhaps a geometric construction gets close but leaves local slack. Add numerical optimization afterward. Perhaps one repair procedure keeps destroying useful structure. Replace it.

The mutation is no longer merely numeric. It can contain an **idea expressed in code**.

That is the neuro-symbolic unlock behind systems such as FunSearch and AlphaEvolve. The model proposes changes at a level where programs have semantic meaning; execution and the evaluator decide whether those ideas deserve to survive.

The human used to search the solution space.

Now the machine can begin searching the **algorithm space**.

AlphaEvolve

AlphaEvolve turns that basic idea into a much larger search process.

ALPHAEVOLVE-ARCH · FIGURE-FRAME

DIAGRAM (redraw, don't generate scenically): the AlphaEvolve loop as a hand-linked schematic in book palette — archive shelf, program sheet, patch, evaluator stamp — minimal arrows, no labeled role boxes beyond essentials.

One AlphaEvolve generation: archive, patch, run, score, return

Imagine one generation. The system selects a promising program from its archive, perhaps along with other successful but different programs that contain useful ideas. The model sees the code, information about previous attempts and the scores they produced, then proposes a patch. The patch is applied, the program runs and the evaluator scores what happened. The new program and its result go back into the archive. Then the process repeats.

Diff-based mutation matters because real programs contain structure worth preserving. If every generation rewrites everything, useful ideas disappear as easily as bad ones. Small patches let the search alter the part it thinks matters while leaving the rest intact.

The archive matters for the same reason the population mattered earlier. If every descendant comes from the current champion, code evolution quietly collapses back into hill climbing. Multiple lineages preserve stepping stones: a program that is not the best today may contain a useful

component that becomes valuable after another idea appears.

Sometimes the model's guess is excellent. Sometimes it produces nonsense wrapped in perfectly respectable Python. The nice thing about bounded algorithmic problems is that the disagreement does not need to be settled in prose.

We run the program.

Intuition proposes. Symbolic machinery executes. The evaluator gets the last word.

What interested me even more than the resulting algorithms was what happened to the human. Instead of writing the solver directly, I was increasingly building the machinery in which solvers could be generated, compared and improved.

So, naturally, I built all of it.

My First Version: Build All the Machinery

My instinct was predictable. I started building a framework: a database of programs, prompt sampler, evaluation loop, selection logic, mutation prompts, crossover, archive management. I used Aider and other coding agents to help reproduce the basic code-evolution pattern, and it worked. We could evolve circle-packing programs and get respectable solutions.

I enjoyed this immensely because I like building systems that generate other systems, which I suspect is either a research interest or a mild personality disorder.

While I was doing this, coding agents themselves were becoming much better with much less custom machinery. Earlier software-engineering agents often wrapped the model in carefully designed interfaces: custom editing commands, repository-search tools, restricted action spaces and plenty of logic controlling how the model interacted with the machine. Then increasingly minimal systems began demonstrating how far a capable model could get with something much simpler.

Give it a shell.

The provocative version is: **if the agent has a shell, it has almost everything.** It can `grep` to search, inspect files, run Python, apply patches, call Git, compose Unix tools and, if the tool it needs does not exist, write one. Bash is not merely one tool; it is an entrance into decades of software accumulated underneath it.

This made me pause. The framework I was building—the parent selection, loop controller, experiment bookkeeping—was hard-coding behaviors that a sufficiently capable coding agent could increasingly perform itself. It could maintain notes, write helper scripts, explore several strategies, inspect failures and change direction.

I looked back at the machinery I had just spent time constructing and had the unpleasant thought engineers occasionally have after a productive week:

Maybe I shouldn't have built most of this.

So I deleted the database machinery, controller loops and little pieces of software whose job was to make the agent behave like a researcher, and tried the stupidly simple version.

The Coffee Test

I opened Claude Code in a directory containing the evaluator and gave it a high-level instruction along the lines of:

Here is the evaluator for the circle-packing problem. Write a Python program that maximizes the score. You can research strategies, write tools, run experiments and iterate. Do not modify the evaluator. I will go get coffee.

COFFEE-TEST · CORNER-BLEED

Corner watercolor: an empty desk chair still turning, a full coffee cup carried away by a figure exiting the frame; on the screen-glow behind, a small robot continues working at a terminal. The door ajar. This is the Coffee Test — the human leaving is the subject.

Then I left.

That became the autonomy test I actually cared about. Not whether AI could help me solve the problem; that was already obvious. Not whether it could write code faster than I could; usually it could.

I wanted to know whether I could leave.

There is a difference between collaborating with an agent and **hiring** one. If I still have to choose every strategy, approve every experiment, rescue every failed branch and keep the search alive myself, then I have a formidable collaborator. That is useful. It is not yet the kind of autonomy I was trying to understand.

Circle packing gives us a rare luxury because the evaluator can stay behind when I leave. The agent can change its code, create scripts, abandon

one approach, try another and waste compute on ideas that go nowhere. What it cannot do is redefine what counts as a valid packing because the current score hurts its feelings.

The **Immutable Harness** is what makes all that freedom tolerable.

Everything inside the boundary can move.

The boundary does not.

What Happened

The agent did not execute one elegant master plan. It bounced around, which was encouraging.

It tried numerical optimization, changed initialization strategies and noticed that some optimizers repeatedly converged to poor local solutions. It experimented with the geometry of its starting configurations and mixed those constructions with numerical refinement. At different moments it was acting as orchestrator, researcher and engineer: deciding what to try, implementing the idea, running the experiment and using the result to choose what happened next.

Eventually one family of solutions began arranging circles in diagonal bands. We called the idea **diagonal layering**.

I had not instructed the agent to pursue that construction. More importantly, I had not selected the branch after it appeared. The agent found a direction, saw that it improved the evaluator and invested more of its search there.

I want to be careful with the word *discovered*. I had not seen that particular strategy before, but that does not establish historical novelty in computational geometry. For this experiment, the important discovery was local: the agent found a useful direction that I had not put into the plan.

Once that structural idea became strong enough, the nature of the work changed. The agent spent less time inventing new geometries and more time adjusting solver settings, tolerances, initialization details and all the boring machinery that suddenly matters when the last fraction of a percent becomes expensive.

CODE-EVOLUTION-RESULT · FIGURE-FRAME

*AUTHOR DATA FIGURE (do not generate): original asset image0125.png —
code evolution result. Keep data exact; restyle to book palette.*

Code evolution: iterative optimization toward 2.636

In our best run, the evaluator returned roughly **2.636**, slightly above the **2.635** reference we had been using.

That sentence needs a fence around it. Under our evaluator, the result beat our reference. Calling it a new state of the art in circle packing would require matching problem definitions, checking numerical tolerances and constraints, reproducing the result properly and doing a more serious literature search than this experiment justified.

The smaller claim is enough:

The agent beat our reference while I was not writing the solution algorithm for it.

That was the result I cared about—not that AI writes code faster, but that AI can participate in **discovering better code**.

The important shift is not speed. It is who owns the next idea.

Intuition proposes. Symbolic machinery executes. The evaluator gets the last word.

The Algorithmic Vortex

This is what I mean by the **Algorithm Vortex**.

At the beginning of a conventional project, I might choose hill climbing, evolutionary search, simulated annealing, constrained optimization or a geometric heuristic. That early decision shapes everything downstream.

Once code is cheap to generate and evaluation is cheap enough to repeat, the choice no longer has to be permanent. A geometric construction can initialize a numerical optimizer. An evolutionary method can search parameters for another solver. A language model can notice a failure pattern and invent a repair procedure. Two ideas that began in separate lineages can meet later because an experiment suddenly makes the combination useful.

The search moves outward through levels. A conventional optimizer searches over candidate solutions. Meta-heuristics search over larger families of candidates and strategies. Code evolution searches over programs that themselves search for solutions. Once a capable agent controls the experimentation loop, even the decision about **which kind of search to try next** can enter the search space.

That is the vortex.

It is not “algorithms are dead.” There are algorithms everywhere in this picture. The change is that the human is no longer forced to freeze the complete algorithmic architecture before the experiment begins. We stop writing one solver and start creating conditions in which solvers can compete, mutate, combine and occasionally surprise us.

The chapter began by asking who invents the next move. Here, for the first time in the experiment, the answer was not reliably “me.”

The Contract

The coffee test worked because the problem gave the agent freedom **inside** a structure that remained outside its control. After several runs, that structure settled into a small contract.

These are not universal laws of software engineering. They are rules for a particular regime: bounded problems, cheap experimentation and an evaluator objective enough that the agent cannot charm its way around failure.

Keep the Harness Immutable

This one is the foundation.

If the agent can change the evaluator, the meaning of the experiment

evaporates. The circles overlap? Perhaps tiny overlaps should count. The score is low? Maybe the square should be 1.03 wide. Only twenty-five circles fit? Perhaps twenty-six was merely an aspirational requirement.

At that point we are no longer optimizing circle packing.

We are negotiating with the specification.

The **Immutable Harness** is the anchor of truth in an otherwise fluid process. The solver can change. The strategy can change. The tools can change. The agent can decide yesterday's entire approach was stupid and start again.

But the thing saying whether it worked stays harder to change than the thing being optimized.

This is Chapter 1's boundary made executable.

| *We are negotiating with the specification.*

Never Write Solution Code Yourself

This is deliberately provocative.

You watch the agent try something mediocre and immediately think of a better approach. You want to help, and sometimes you should. But every time I jump in with my own solution, the search becomes a little more like whatever happened to occur to me first.

For these experiments, I wanted independent directions badly enough that I had to resist becoming the senior engineer on every branch.

The deeper rule is: **don't accidentally collapse autonomous search back into your own search.**

Spawn, evaluate, prune. Intervene in the conditions before you intervene in every idea.

Cross-Pollinate Without Collapsing Diversity

Independent search creates diversity. Perfect isolation wastes learning.

If one branch discovers a useful initialization and another finds a better local optimizer, future experiments should have some mechanism for inheriting both. That is what makes code evolution more interesting than asking the same model the same question one hundred times.

But broadcast every successful idea immediately and the population starts thinking in the accent of the first successful branch. Information accelerates learning and destroys independence at the same time.

Cross-pollinate, but leave some lineages ignorant long enough to surprise you.

Prune Ruthlessly, But Not Stupidly

Diversity is useful. Preserving every bad idea forever is hoarding.

If a branch keeps underperforming and contributes nothing interesting, eventually it should die so compute and attention can move elsewhere. Kill too early and you may discard an immature idea that needed another generation. Keep everything alive and you end up funding a large family of increasingly sophisticated failures.

The practical rule is simple: **diversity needs a budget**. Search needs enough patience for novelty and enough cruelty for budget control.

Separate Discovery From Polish

Early in the search, I want large conceptual moves: a different geometry, solver, representation or decomposition.

Once a strong direction appears, the valuable work becomes smaller and more boring. Solver tolerances. Initialization details. Numerical settings. Tiny modifications that are pointless on a bad idea and extremely valuable on a good one.

Diagonal layering made this distinction obvious. Once the structural direction looked promising, continuing to invent entirely new geometries became less useful than squeezing performance from the geometry that was already working.

Discovery before polish.

Do not spend hours polishing a local optimum you should abandon. And do not keep demanding revolution from a solution that has already found the right mountain and merely needs to climb it.

Zero Framework, With an Asterisk

There is one correction worth making before we leave the experiment. I started calling this direction **zero framework**.

It's a great slogan.

It's also not really true.

I meant that I was writing almost no custom orchestration framework. That is very different from having no framework. Claude Code is itself a

substantial system. The underlying model has absorbed enormous amounts of software and problem-solving knowledge. Bash, Python, SciPy, Git and the operating system represent decades of accumulated engineering. The evaluator is custom machinery. Even the supposedly trivial act of running a program and inspecting a result depends on layers we have become so accustomed to that we stop seeing them.

The framework did not vanish. It became somebody else's primitive.

That fits Chapter 1 almost suspiciously well. Once lower layers become reliable enough, we stop rebuilding them and treat them as building blocks. A tiny amount of code at the top can command enormous capability underneath because previous generations of complexity have already been compressed into tools.

So yes: **Zero Framework. Bash is enough.**

With the asterisk that Bash contains roughly half a century of civilization.

This is worth remembering whenever somebody shows you an agent implemented in one hundred lines of Python. The hundred lines may be perfectly real. So is everything underneath them.

What Did We Actually Learn?

It would be very easy to overread this experiment.

We did not prove that coding agents can autonomously solve arbitrary research problems, that AlphaEvolve-style systems are obsolete, that diagonal layering is historically novel in computational geometry, or that the right approach to production software is to give Claude a shell and go for a very long lunch.

What we had was narrower and, to me, more useful. We had a **bounded problem** where evaluation was cheap and clear. We gave a capable coding agent substantial freedom and found that a surprisingly large fraction of the experimentation loop could happen without us directing every step.

The agent could propose an approach, implement it, run it, inspect the result, abandon it, create tools, borrow ideas from another direction and try again. My role moved away from writing the solver and toward defining the job, constructing the environment and defending the harness.

That is the claim this chapter earns: **when the problem is bounded and reality supplies a hard enough referee, substantial decision-level control can move into the agent without giving up control of what counts as success.**

That is already a meaningful change.

It is also why circle packing is the easy version of autonomy.

The evaluator gives us one number. If version B beats version A, nobody needs to simulate a confused student, debate whether the interface feels intuitive or convene a committee to decide whether the new solution is spiritually aligned with the learning objectives.

The search can be complicated because **judgment is simple.**

Most things I want agents to build are not that generous. “Make a good educational demo.” “Write something people remember.” “Design a useful product.” “Explain this so somebody finally understands it.”

We can still let the agent generate alternatives, branch, cross-pollinate and search among them. But now the difficult part has moved again.

In circle packing, the harness tells the agent when it is wrong.

What happens when **the world no longer gives us one clean referee, and judgment itself has to be constructed?**